

cmaes : A Simple yet Practical Python Library for CMA-ES

Masahiro Nomura*
nomura_masahiro@cyberagent.co.jp
CyberAgent
Shibuya, Tokyo, Japan

Masashi Shibata*
mshibata@preferred.jp
Preferred Networks
Chiyoda, Tokyo, Japan

ABSTRACT

The covariance matrix adaptation evolution strategy (CMA-ES) has been highly effective in black-box continuous optimization, as demonstrated by its success in both benchmark problems and various real-world applications. To address the need for an accessible yet potent tool in this domain, we developed *cmaes*, a simple and practical Python library for CMA-ES. *cmaes* is characterized by its simplicity, offering intuitive use and high code readability. This makes it suitable for quickly using CMA-ES, as well as for educational purposes and seamless integration into other libraries. Despite its simplistic design, *cmaes* maintains enhanced functionality. It incorporates recent advancements in CMA-ES, such as learning rate adaptation for challenging scenarios, transfer learning, and mixed-integer optimization capabilities. These advanced features are accessible through a user-friendly API, ensuring that *cmaes* can be easily adopted in practical applications. We regard *cmaes* as the first choice for a Python CMA-ES library among practitioners. The software is available under the MIT license at <https://github.com/CyberAgentAILab/cmaes>.

CCS CONCEPTS

• Mathematics of computing → Continuous optimization.

KEYWORDS

covariance matrix adaptation evolution strategy, black-box optimization, Python library

ACM Reference Format:

Masahiro Nomura* and Masashi Shibata*. 2024. *cmaes* : A Simple yet Practical Python Library for CMA-ES. In *Proceedings of Preprint*. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Black-box optimization focuses on optimizing an objective function whose internal structure is either unknown or inaccessible. In this approach, the objective function is regarded as a “black-box”: while it provides outputs in response to given inputs, the internal process generating these outputs remains concealed during optimization. This is invaluable when the objective function is complex, costly to evaluate, or lacks an explicit analytical form. The primary challenge in black-box optimization is enhancing the quality of the objective function value with minimal evaluations.

The covariance matrix adaptation evolution strategy (CMA-ES) [22, 28] stands out as a method for solving black-box continuous

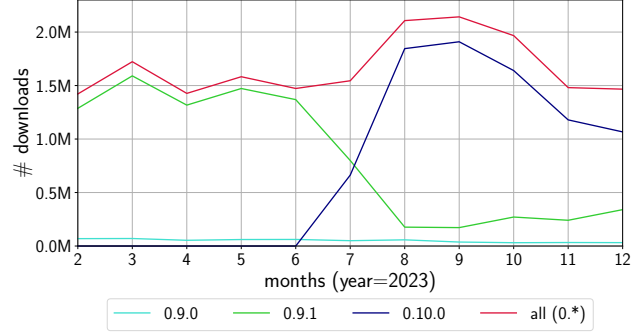


Figure 1: Our library’s monthly download numbers have surpassed one million. Statistics are retrieved from PePy [45]. Different colors represent different versions, with red indicating download numbers across all versions.

optimization problems. It optimizes by sampling candidate solutions from a multivariate Gaussian distribution, enabling parallel solution evaluations and showcasing significant parallelism. Its superiority is evident compared to other black-box optimization methods, particularly in challenging scenarios such as ill-conditioned, non-separable, or rugged problems [43]. The effectiveness of CMA-ES spans various real-world applications, including computer vision [30, 31], natural language processing [46, 47], reinforcement learning [18], and automated machine learning [39, 40].

In this work, we introduce *cmaes*, a Python-implemented CMA-ES library rapidly gaining popularity in the field. This is evidenced by over 250 GitHub stars and its integration into renowned Python libraries such as Optuna [1] and Katib [16]. Its popularity is further highlighted by approximately one million monthly downloads, as shown in Figure 1.

The cornerstone of *cmaes* lies in its *simplicity* and *practicality*. To ensure *simplicity*, our focus has been on high code readability, making it an inspiration for other evolution strategy libraries such as *evojax* [48] and *evosax* [32], and an excellent educational resource. Regarding its *practicality*, *cmaes* stands out by incorporating recent significant advancements in CMA-ES, setting it apart from other libraries. This integration includes (i) automatic learning rate adaptation for solving *difficult* problems such as multimodal and noisy environments *without* expensive hyperparameter tuning, (ii) transfer learning, and (iii) mixed-integer optimization. These advanced features are easily accessible through user-friendly APIs, establishing *cmaes* as the go-to choice for practitioners.

In this paper, we aim to provide a comprehensive overview of *cmaes*’s motivation and distinctive features, making this paper a valuable guide for practitioners to effectively use the library. The remainder of this section discusses related work of Python software

* These authors contributed equally. This work was conducted while Masashi Shibata was affiliated with CyberAgent.

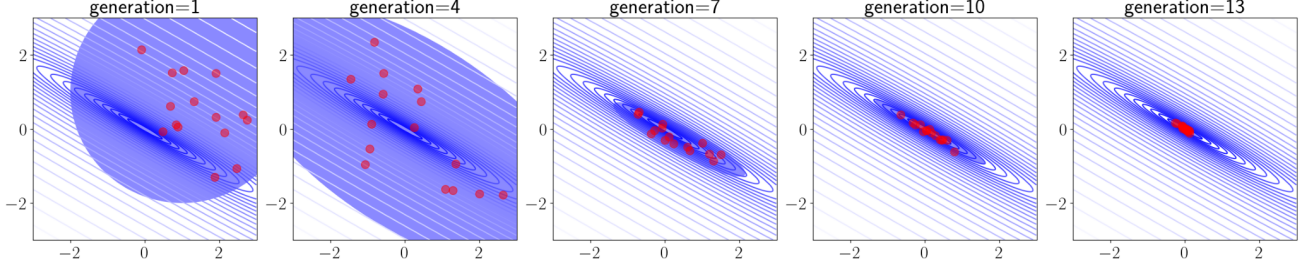


Figure 2: Example of CMA-ES optimizing $f(x) = f_{\text{Ellipsoid}}(Rx)$, where $f_{\text{Ellipsoid}}(x) = x_1^2 + (10x_2)^2$ and $R \in \mathbb{R}^{2 \times 2}$ is a rotation matrix rotating $\pi/6$ around the origin. Population size $\lambda = 15$ and the initial distribution $m^{(0)} = [1.0, 1.0]$, $\sigma^{(0)} = 1.0$, $C^{(0)} = I$. Ellipse represents the distribution of CMA-ES and the red points represent the sampled solutions. CMA-ES efficiently addresses ill-conditioned and non-separable problems by adapting the distribution parameters.

for CMA-ES. Section 2 explains the specifics of CMA-ES adopted in this study. Section 3 discusses the design philosophy behind `cmaes`, provides basic usage information, and discusses the software aspects of our library. Section 4 discusses several recent advances in the field of CMA-ES that can be used through `cmaes`. Section 5 concludes the paper with a summary and discussion.

Related Work. While there are several sophisticated libraries for CMA-ES, `pycma` [24] is particularly renowned for its comprehensive features and detailed documentation. Indeed, `pycma` includes several features that our library lacks, such as handling of non-linear constraints [7], sophisticated restriction of the covariance matrix [2–4], and surrogate-assisted methods [23]. However, it may pose challenges for users seeking an in-depth understanding due to its complexity. In contrast, our library `cmaes` focuses on basic and essential features, with an emphasis on simplicity and ease of understanding, appealing to practitioners who prefer straightforward implementations. This simplicity also enhances the library’s flexibility for integration of `cmaes` with other libraries.

Beyond `pycma`, the Python ecosystem houses other notable CMA-ES implementations. For example, `evobjax` [48] and `evosax` [32] are JAX-based libraries, ideal for leveraging GPUs or TPUs in scalable optimization. `pymoo` [10] specializes in multi-objective optimization, an area not covered by our library. Additionally, `Nevergrad` [42] offers a diverse range of optimization methods, including evolution strategies and Bayesian optimization, facilitating comparative studies between methods such as CMA-ES and Bayesian optimization [15]. This variety underscores the rich landscape of optimization tools available, each with its unique advantages and specialized applications.

2 CMA-ES

We consider minimization¹ of the objective function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. CMA-ES optimizes f by sampling solutions from a multivariate Gaussian distribution $\mathcal{N}(m, \sigma^2 C)$, wherein $m \in \mathbb{R}^d$ is the mean vector, $\sigma \in \mathbb{R}_{>0}$ is the step-size, and $C \in \mathbb{R}^{d \times d}$ is the covariance matrix. Following the evaluation of solutions for f , CMA-ES updates the distribution parameters m , σ , and C to produce more promising solutions. Figure 2 provides an example of optimization using

CMA-ES. In this paper, we present a version of Ref. [22] that our library `cmaes` adopts. For a detailed explanation of CMA-ES, see the Hansen’s excellent tutorial [22].

CMA-ES begins by initializing the parameters $m^{(0)}$, $\sigma^{(0)}$, and $C^{(0)}$ parameters. The following steps are then repeated until the stopping condition is met:

Step 1. Sampling and Ranking

In the $g + 1^{\text{st}}$ generation (with g starting from 0), for the population size λ , λ candidate solutions x_i ($i = 1, 2, \dots, \lambda$) are independently sampled from $\mathcal{N}(m^{(g)}, (\sigma^{(g)})^2 C^{(g)})$:

$$y_i = \sqrt{C^{(g)}} z_i, \quad (1)$$

$$x_i = m^{(g)} + \sigma^{(g)} y_i, \quad (2)$$

where $z_i \sim \mathcal{N}(0, I)$ and I is the identity matrix. These solutions are evaluated using the function f and sorted in ascending order. We denote $x_{i:\lambda}$ as the i -th best candidate solution $f(x_{1:\lambda}) \leq f(x_{2:\lambda}) \leq \dots \leq f(x_{\lambda:\lambda})$. Additionally, $y_{i:\lambda}$ represents the random vectors corresponding to $x_{i:\lambda}$.

Step 2. Update Evolution Path

Using the parent number $\mu \leq \lambda$ and the weights w_i , where $w_1 \geq w_2 \geq \dots \geq w_\mu > 0$ and $\sum_{i=1}^\mu w_i = 1$, the weighted average $dy = \sum_{i=1}^\mu w_i y_{i:\lambda}$ is calculated. The evolution paths are then updated by:

$$p_\sigma^{(g+1)} = (1 - c_\sigma) p_\sigma^{(g)} + \sqrt{c_\sigma(2 - c_\sigma) \mu_w} \sqrt{C^{(g)}}^{-1} dy, \quad (3)$$

$$p_c^{(g+1)} = (1 - c_c) p_c^{(g)} + h_\sigma^{(g+1)} \sqrt{c_c(2 - c_c) \mu_w} dy, \quad (4)$$

where $\mu_w = 1 / \sum_{i=1}^\mu w_i^2$, c_σ , and c_c are the cumulation factors, and $h_\sigma^{(g+1)}$ is the Heaviside function,

$$h_\sigma^{(g+1)} = \begin{cases} 1 & \text{if } \frac{\|p_\sigma^{(g+1)}\|}{\sqrt{1 - (1 - c_\sigma)^{2(g+1)}} \mathbb{E}[\|\mathcal{N}(0, I)\|]} < 1.4 + \frac{2}{d+1}, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

where $\mathbb{E}[\|\mathcal{N}(0, I)\|] \approx \sqrt{d} \left(1 - \frac{1}{4d} + \frac{1}{21d^2}\right)$ is the expected norm of a sample of a standard Gaussian distribution.

¹This is w.l.o.g. because replacing f with $-f$ allows to consider maximization.

Step 3. Update Distribution Parameters

The distribution parameters are updated as follows:

$$m^{(g+1)} = m^{(g)} + c_m \sigma^{(g)} dy, \quad (6)$$

$$\sigma^{(g+1)} = \sigma^{(g)} \exp \left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|p_\sigma^{(g+1)}\|}{\mathbb{E}[\|\mathcal{N}(0, I)\|]} - 1 \right) \right), \quad (7)$$

$$C^{(g+1)} = \left(1 + (1 - h_\sigma^{(g+1)}) c_1 c_c (2 - c_c) \right) C^{(g)} + c_1 \underbrace{\left[p_c^{(g+1)} \left(p_c^{(g+1)} \right)^\top - C^{(g)} \right]}_{\text{rank-one update}} + c_\mu \underbrace{\left[\sum_{i=1}^{\lambda} \left(w_i^\circ y_{i:\lambda} y_{i:\lambda}^\top - w_i C^{(g)} \right) \right]}_{\text{rank-}\mu \text{ update}}, \quad (8)$$

where $w_i^\circ := w_i \cdot (1 \text{ if } w_i \geq 0 \text{ else } d / \|\sqrt{C^{(g)}}^{-1} y_{i:\lambda}\|^2)$, c_m is the learning rate for m ; c_1 and c_μ are the learning rates for the rank-one and $-\mu$ updates of C , respectively; and d_σ is the damping factor for the σ adaptation. This adaptation of σ is known as the cumulative step-size adaptation (CSA) [28]. The update of distribution parameters in CMA-ES is closely related to the natural gradient descent [5].

CMA-ES is invariant to order-preserving transformations of the objective function, utilizing solution rankings instead of actual values. Additionally, it possesses affine invariance within the search space. These features enable the generalization of its empirical successes to a wider range of problems [25].

3 CMAES : SIMPLICITY & PRACTICALITY

This section aims to offer a comprehensive explanation of the various software aspects of our library. Initially, we delve into the core design philosophy of our library in Section 3.1, where we outline the fundamental principles that have shaped its development. Following this, Section 3.2 provides guidance on the installation process and the basic usage of the library, ensuring a smooth start for new users. We then shift our focus to features specifically designed to enhance the software’s quality: these include the continuous integration pipeline discussed in Section 3.3, the methodology for identifying unexpected errors via fuzzing in Section 3.4, and the animated visualization tools in Section 3.5. To demonstrate the practical application of `cmaes` in other libraries, Section 3.6 presents a use case involving Optuna [1], showcasing the integration and utility of `cmaes` in a real-world context.

3.1 Design Philosophy

Our ultimate goal is to develop a Python CMA-ES library that becomes the first choice for practitioners. To achieve this, the library is designed with the dual aim of being both *simple* and *practical*, whose the detailed motivation is presented below.

Simplicity. To achieve our goal, the library is designed with utmost simplicity. We identify three major benefits of this approach: Firstly, simplicity enhances software quality, easing debugging and maintenance, which is crucial for long-term reliability and practical use. Simplified maintenance also invites more feedback from the developer community, leading to a more refined product. Secondly,

a simpler library is inherently more user-friendly. A straightforward API makes it easier for users to understand and utilize the library effectively. Lastly, from an educational standpoint, a simpler implementation aids in understanding the code in relation to the algorithm. This makes the library accessible not only to knowledgeable practitioners but also to beginners eager to learn about CMA-ES.

Practicality. While we prioritize keeping the library simple, it is essential to develop a library that solves the real-world challenges practitioners face. To maximize the library’s practicality, it is important to integrate insights from cutting-edge research on CMA-ES, particularly those with a strong practical motivation. However, it is equally important to ensure the library is user-friendly, as not all practitioners are knowledgeable about CMA-ES. The challenge we address is ensuring the practicality while maintaining the simplicity of our library.

Our Library. Based on our discussions, we have designed the library to be both simple and practical:

To ensure **simplicity**, our primary focus has been on achieving high code readability in the implementation. Although the code is not included in this paper due to space limitations, renowned evolution strategy libraries such as `evobjax` [48] and `evosax` [32] are inspired by our implementation.² This demonstrates the readability of our code and then its value for educational purposes. Our library has introduced animated visualizations (Section 3.5) that facilitates more an intuitive understanding of the behavior. In addition to the simplicity of the code itself, our APIs are also simple and easy-to-use, which will be presented in Section 3.2. To further be able to continuously keep the software quality, we have introduced continuous integration services (Section 3.3) and unexpected error identification testing via Fuzzing (Section 3.4).

To enhance **practicality**, our library has implemented three methods that are highly relevant in practical scenarios. The first method is for solving difficult problems such as multimodal and noisy issues *without* the need for expensive hyperparameter tuning. The second method is a transfer learning that accelerates CMA-ES by utilizing previous optimization results. **The third method is for handling mixed-integer variables rather than only continuous variables. The details are explained in Section 4.** To our knowledge, these methods have not yet been implemented into other evolution strategy libraries. Therefore, their presence in our library represents a distinctive feature. Importantly, these methods are generalizations of CMA-ES outlined in Section 2, and then do not require changes to the original CMA-ES implementation. Consequently, our library retains its simplicity. The API has also been designed for maximum simplicity, as will be detailed in Section 4.

3.2 Getting Started

Installation. `cmaes` is available on the Python Packaging Index (PyPI) [13] and can be easily installed using the following command. Our library, written in pure Python and depending solely

²https://github.com/google/evobjax/blob/67c90e1/evobjax/algo/cma_jax.py#L4-L6 and https://github.com/RobertTLange/evosax/blob/67b361/evosax/strategies/cma_es.py#L97-L99

on NumPy [29], is straightforward to install in a variety of user environments.

```
1 $ pip install cmaes
```

For Anaconda users, the following command is recommended:

```
1 $ conda install -c conda-forge cmaes
```

Basic Usage. To illustrate its basic functionality, Listing 1 shows a simple example using `cmaes`. The library employs an ask-and-tell interface [11]. After initializing `CMA(·)` with initial `mean` and `sigma`, the process involves the following steps: (1) Candidate solutions are generated using the `ask()` method. (2) The sampled solutions are then evaluated by an objective function. (3) The distribution parameters are updated based on the solutions and their objective function values using the `tell(·)` method.

```
1 import numpy as np
2 from cmaes import CMA
3
4 def objective(x):
5     # The best parameters are x = [3, -2]
6     return (x[0] - 3) ** 2 + (10 * (x[1] + 2)) ** 2
7
8 optimizer = CMA(mean=np.zeros(2), sigma=2)
9 for generation in range(100):
10     solutions = []
11     for _ in range(optimizer.population_size):
12         x = optimizer.ask()
13         value = objective(x)
14         solutions.append((x, value))
15
16     print(f"{generation=} {value=} {x=}")
17     optimizer.tell(solutions)
```

Listing 1: Simple example code with `cmaes`.

This ask-and-tell interface, by decoupling the optimizer from the objective function, adds flexibility to the library. For example, it is easy to implement CMA-ES with restart strategies, such as IPOP-CMA-ES [8] or BIPOP-CMA-ES [21]. Listing 2 provides an example code for IPOP-CMA-ES. In this example, the algorithm restarts with a new mean vector and a doubled population size when the stopping conditions are met (i.e., when `should_stop()` returns True). These stopping conditions are based on those used in `pycma` [24]. Additionally, we can specify box constraints by adding the argument, for example, `bounds=np.array([-3, 3], [-3, 3])` to `CMA(·)`; if a solution is not feasible, a resampling method is used, which is reasonable when the optimum solution is not very close to the constraint.

```
1 import numpy as np
2 from cmaes import CMA
3
4 popsize = 6
5 optimizer = CMA(mean=np.zeros(2), sigma=3,
6                 population_size=popsize)
7
8 for generation in range(500):
9     solutions = []
10    for _ in range(optimizer.population_size):
11        x = optimizer.ask()
12        value = objective(x) # objective is omitted here
13        solutions.append((x, value))
14    optimizer.tell(solutions)
15
16    if optimizer.should_stop():
```

```
16 # Increase the popsize with each restart
17 popsize = popsize * 2
18 # Randomize the initial mean
19 optimizer = CMA(
20     mean=np.random.uniform(-5, 5, 2),
21     population_size=popsize, sigma=3
22 )
```

Listing 2: Example code for IPOP-CMA-ES.

3.3 Continuous Integration Pipeline

To maintain the quality of the library, we have utilized a continuous integration (CI) service. These services automatically activate in response to GitHub pull requests, facilitating software quality management without impeding the development pace. To achieve this, we have introduced several components into the development process: We perform tests across multiple Python versions to ensure compatibility. We employ static analysis tools for maintaining code quality. This includes Flake8 [9] for code style, Black [14] for code formatting, and Mypy [41] for type checking.

In addition to these basic validations, our repository conducts quick benchmarking, as illustrated in Figure 3. The aim of this quick benchmarking is to ensure that no significant bugs are introduced by recent changes. This benchmarking is to confirm whether the performance of `cmaes` is not quite different to that of `pycma`. For this, we use `kurobako` [49] to create benchmark results efficiently. The benchmark results are uploaded onto Google Cloud Storage³ and subsequently incorporated into the corresponding pull request as comments, including images and descriptions. This integration is facilitated by custom GitHub Actions⁴.

It is important to acknowledge that the goal of our benchmarking process is to quickly identify evident bugs in the implementation *without* incurring substantial computational costs, rather than conducting rigorous benchmarking. For more comprehensive benchmarking, we recommend employing specialized software such as COCO [26], a platform for comparing continuous optimizers in a black-box setting.

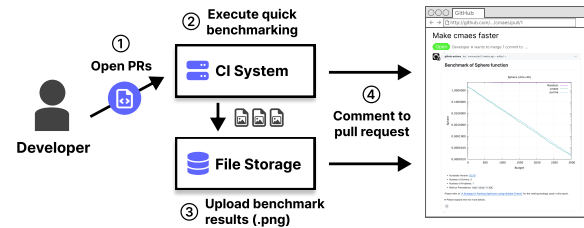


Figure 3: System diagram for CI pipeline.

3.4 Unexpected Error Identification via Fuzzing

To enhance software reliability, it is essential to proactively address unexpected error terminations. These errors are often caused by unforeseen user inputs. However, anticipating and listing all possible unexpected scenarios manually is a challenging and labor-intensive

³<https://cloud.google.com/storage/>

⁴<https://github.com/features/actions/>

task. This highlights the importance of automated methods, such as fuzzing, in simplifying this process.

Fuzzing is an automated software testing method aimed at discovering unexpected inputs that could cause program crashes. It involves generating a wide range of inputs. The key component in this process is the fuzzing engine, which automatically generates various inputs for the target program. A notable technique is *coverage-guided fuzzing*. This approach not only generates random inputs but also collects code coverage data. When an input is found to increase the code coverage, indicating untested paths, this information is used to methodically explore different branches within the program's code. This strategy enhances the detection of unexpected errors, significantly contributing to the reliability of the software.

Listing 3 shows an example of fuzzing code used in our library. In this example, to identify inputs that could lead to unexpected errors, fuzzing generates various inputs. These include evaluated solutions (`tell_solutions`) for updating the distribution, the number of dimensions (`dim`), and the distribution parameters (`mean` and `sigma`). To perform the coverage-guided fuzzing more efficiently, especially with the generation of NumPy arrays, we utilized Atheris [17] in combination with Hypothesis [33].

```

1 import atheris
2 from hypothesis import given, strategies as st
3 import hypothesis.extra.numpy as npst
4
5 @given(data=st.data())
6 def fuzzing_cmaes(data):
7     # Generating random parameters.
8     dim = data.draw(st.integers(min_value=2, ...))
9     n_iterations = data.draw(st.integers(min_value=1))
10    mean = data.draw(npst.arrays(shape=dim, dtype=float))
11    sigma = data.draw(st.floats(min_value=1e-16))
12
13    optimizer = CMA(mean, sigma)
14    for _ in range(n_iterations):
15        # Checking "ask" function works properly
16        optimizer.ask()
17
18        tell_solutions = data.draw(st.lists(...))
19        try:
20            optimizer.tell(tell_solutions)
21        except AssertionError:
22            # Skip AssertionError exceptions, that are
23            # raised if our generated data is invalid
24            return
25
26 atheris.Setup(...)
27 atheris.Fuzz() # Brute forcing until the target crashes

```

Listing 3: Example code of fuzzing for finding the inputs that cause unexpected errors.

3.5 Animated Visualization

We recognize the value of intuitively understanding the behavior of CMA-ES before delving into its detailed mathematical equations. To achieve this, our library provides animated visualizations that illustrate the changes in the CMA-ES's multivariate Gaussian distribution.

Figure 4 shows an example of the animated visualization for a single iteration. Note that the actual output is an animated GIF, not a static figure, allowing for a dynamic observation of how CMA-ES progresses and converges towards the optimum. This animated

visualization aids in understanding the effect of various operations in CMA-ES, such as the rank-one update and step-size adaptation, in a visually interpretable way. It is also beneficial for intuitively comparing behaviors across different optimization methods implemented in our library, such as separable CMA-ES [44] and natural evolution strategies [52]. Consequently, animated visualization also serves as a useful tool for verifying the expected behavior of the implemented algorithms.

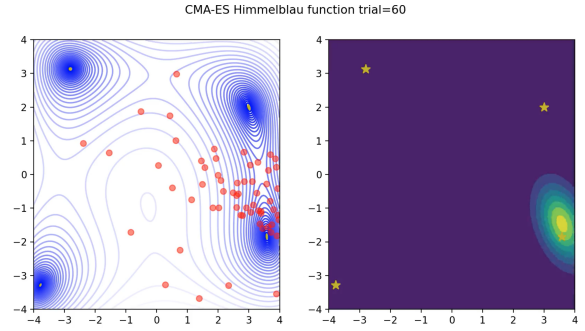


Figure 4: Output example of animated visualization. (Left) Contour lines and sampled solutions. (Right) Multivariate Gaussian distribution in CMA-ES.

3.6 Integration with Real-World Systems: Use Case from Optuna

To exemplify the seamless integration capabilities of our library with other libraries, we present a use case involving Optuna [1], a renowned library extensively utilized for hyperparameter optimization of machine learning algorithms. An example code of CMA-ES via Optuna is shown in Listing 4, where `CmaEsSampler` internally utilizes our library.

```

1 import optuna
2
3 def objective(trial: optuna.Trial) -> float:
4     x1 = trial.suggest_float("x1", -4, 4)
5     x2 = trial.suggest_float("x2", -4, 4)
6     return (x1 - 3) ** 2 + (10 * (x2 + 2)) ** 2
7
8 if __name__ == "__main__":
9     sampler = optuna.samplers.CmaEsSampler()
10    study = optuna.create_study(sampler=sampler, storage=
11        "sqlite:///optuna.db")
12    study.optimize(objective, n_trials=250)

```

Listing 4: Example code of Optuna with CmaEsSampler.

A key factor in Optuna's integration of `cmaes` is not only its clean code and simple design but also the reduced size of its pickle serialization. In hyperparameter optimization, the focus of Optuna, unexpected errors may occur during the training of machine learning models. Consequently, it is crucial to make the optimization process resumable, necessitating the regular saving of CMA-ES optimization data, including distribution parameters. Therefore, reducing the pickle size is critical for Optuna. Our library has introduced specialized `__getstate__` and `__setstate__` methods that are invoked during pickle serialization and deserialization.

These modifications have successfully achieved further lightweighting of the pickle objects. Figure 5 shows the significant reduction of the serialization size for the CMA-ES objects using pickle in `cmaes`, compared to `pycma`.

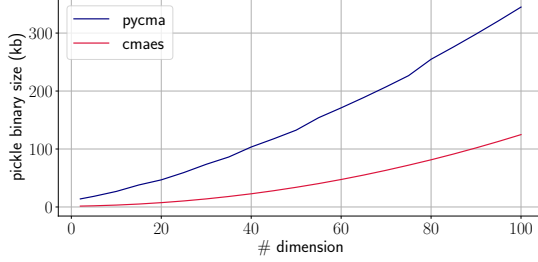


Figure 5: Comparison of the serialization size for the CMA-ES objects using pickle in `pycma` and `cmaes`.

4 EXAMPLES WITH RECENT ADVANCES

A key aspect of our library is offering highly practical methods recently proposed in the context of CMA-ES through easy-to-use APIs. In this section, we will highlight the following methods as examples, providing brief explanations and usage for each.

- **LRA-CMA** (presented at GECCO’23 and nominated for the best paper in the ENUM Track) [37]: LRA-CMA can solve multimodal and noisy problems *without* expensive hyperparameter tuning.
- **WS-CMA** (presented at AAAI’21) [39]: WS-CMA can solve a current task efficiently by utilizing results of a similar task and setting good initial distributions (i.e. transfer learning).
- **CMAwM** (presented at GECCO’22 and nominated for the best paper in the ENUM Track) [19]: CMAwM can stably solve mixed-integer problems by modifying distribution parameters so that generated samples are not fixed.

Importantly, these methods are a generalization of CMA-ES, which means that they do not change the original implementation of CMA-ES. This enables the preservation of the library’s simplicity, making it easier for those who initially learned CMA-ES to straightforwardly delve into understanding these methods.

4.1 Multimodal and Noisy Problems

Why do we need LRA-CMA? The *multimodality* of the objective function is one of the most important properties that make optimization difficult. Figure 7(a) shows the landscape of the Rastrigin function as an example of multimodal problems. Due to the presence of numerous local optima in this function, CMA-ES may sometimes converge towards an incorrect solution. A common practice to address these issues is to increase population size λ [27]; however, the hyperparameter tuning to determine the appropriate value for λ often requires a significant amount of computational time and resources.

Another difficulty arising in practical applications is the presence of *noise*, where the observed objective function value is obtained as $y = f(x) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_n^2)$, with σ_n^2 representing the noise variance. In this case, unless an appropriate λ is chosen in accordance with

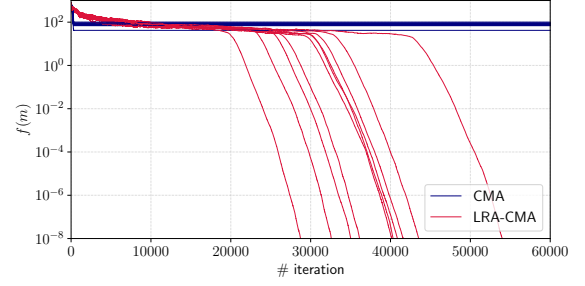


Figure 6: Comparison between vanilla CMA-ES and LRA-CMA on the 40-D Rastrigin function. Initial distribution is $m^{(0)} = [3.0, \dots, 3.0]$, $\sigma^{(0)} = 2.0$, $C^{(0)} = I$. Population size is set as the recommended value, i.e., $\lambda = 4 + \lfloor 3 \log(d) \rfloor = 15$. Vanilla CMA-ES gets trapped in local optima, whereas LRA-CMA succeeds in finding global optima *without* any hyperparameter tuning.

the scale of the noise, attaining a high-quality objective function value becomes challenging.

Recognizing that increasing λ has an effect similar to decreasing η [35], our library employs automatic adaptation of the learning rate of the distribution parameters, i.e., η , rather than adaptation of the population size λ [36] from the practical standpoint: Practitioners often prefer to set λ to a specific number of workers to prevent resource waste. However, λ adaptation might not always use all computing resources due to the variability of λ during optimization. In contrast, η adaptation allows for maximal resource utilization by fixing λ at the highest number of available workers.

What is LRA-CMA? Learning Rate Adaptation (LRA-)CMA [37, 38] automatically adjusts η to maintain a constant signal-to-noise ratio (SNR). Therefore, η decreases when SNR is low and increases when SNR is high, allowing for a responsive adaptation of η to the search difficulty. The validity of using SNR is discussed theoretically in [38]. Figure 6 shows the results of multiple trials on the Rastrigin function. LRA-CMA, which includes η adaptation, consistently finds the global optimum without any hyperparameter tuning, unlike the vanilla CMA-ES, which often gets trapped in local optima.

Notably, LRA-CMA is also effective in noisy environments. This is based on the observation that noisy problems can be viewed as multimodal problems from an optimizer’s perspective, as illustrated in Figure 7. Indeed, LRA-CMA-ES has been observed to simultaneously address noise and multimodality issues effectively [37].

How to use? Users can run LRA-CMA simply by adding the argument `lr_adapt=True` during the initialization of `CMA(·)`.

4.2 Transfer Learning

Why do we need WS-CMA? Practically it is possible that we have already solved a problem (*source task*) related to the problem we are currently trying to solve (*target task*). In such cases, it would be wasteful to optimize the target task from scratch without utilizing the results of the source task. Therefore, it is expected that optimizing the target task can be made more efficient by transferring knowledge from the source task. This knowledge transfer

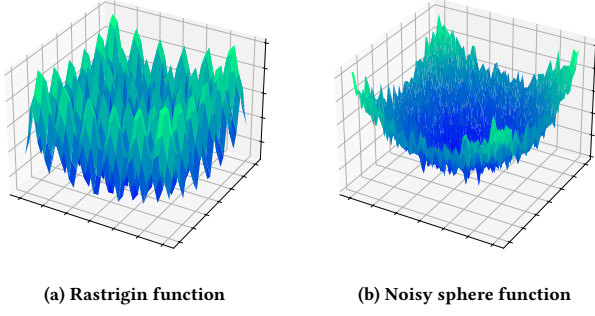


Figure 7: Visualizations of function landscape. In noisy problems, although function value of each point is technically random variable, it can be treated as scalar value from optimizer’s perspective in continuous space.

is especially beneficial when evaluating the objective function is expensive, such as in hyperparameter optimization [12].

What is WS-CMA? Warm Starting CMA-ES (WS-CMA) [39] accelerates the optimization of the target task by utilizing information from the source task. The process of WS-CMA is actually quite simple: Now we assume that there are solutions evaluated on the source task (Figure 8(a)). Based on the results of the source task, WS-CMA first estimates a promising distribution as Gaussian mixture models (GMM) (Figure 8(b)). It then determines the parameters of the initial distribution to fit the promising distribution by minimizing the Kullback-Leibler divergence between GMM and the initial distribution (Figure 8(c)). Subsequently, WS-CMA executes CMA-ES using the determined initial distribution to efficiently solve the target task.

How to use? Listing 5 shows an example code using WS-CMA. We first need to prepare an array (`source_solutions` in the code) whose element is a tuple $(x, f(x))$. By using the method `get_warm_start_mgd()` with the created `source_solutions`, we can obtain the *promising* distribution parameters of the mean vector, the step-size, and the covariance matrix. WS-CMA is then performed by initializing `CMA()` with these parameters.

```

1 import numpy as np
2 from cmaes import CMA, get_warm_start_mgd
3
4 def source_task(x1: float, x2: float) -> float:
5     return x1 ** 2 + x2 ** 2
6
7 def main() -> None:
8     # Generate solutions from a source task
9     # Please replace with your specific task when using
10    source_solutions = []
11    for _ in range(100):
12        x = np.random.random(2)
13        value = source_task(x[0], x[1])
14        source_solutions.append((x, value))
15
16    # Estimate a promising distribution
17    ws_mean, ws_sigma, ws_cov = get_warm_start_mgd(
18        source_solutions)
19    optimizer = CMA(mean=ws_mean, sigma=ws_sigma, cov=
20    ws_cov)

```

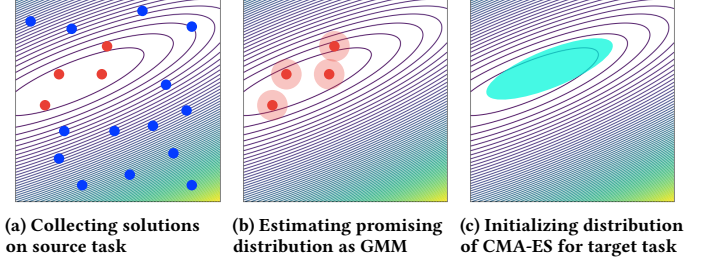


Figure 8: Procedure of WS-CMA.

Listing 5: Example code of WS-CMA.

4.3 Mixed-Integer Optimization

Why do we need CMAwM? While CMA-ES is basically performed for optimization of continuous variables, there are a variety of real-world problems that involve discrete variables such as binary or integers [12, 34, 53]. This raises the question: Can CMA-ES be applied to optimization problems involving a mix of continuous, binary, and integer variables?

A straightforward way to handle discrete variables in CMA-ES is to map continuous variables to discrete ones, i.e., *encoding*. For example, binary variables can be mapped as follows: $x_i (\in \mathbb{R}) \leq 0.5$ is mapped to $0 \in \{0, 1\}$, and $x_i (\in \mathbb{R}) > 0.5$ to $1 \in \{0, 1\}$. However, this naïve approach may bias the search toward only 0s or 1s, potentially leading to stagnation (Figure 9(a)).

What is CMAwM? Our library implements CMA-ES with Margin (CMAwM) [19, 20], a method developed for mixed-integer optimization and has been rapidly gaining attention recently [6, 50, 51]. To address the issue of the above naïve approach, CMAwM corrects distribution parameters to ensure that the probabilities of sampling 0 and 1 remains above a certain threshold (Figure 9(b)). This approach is also applicable to integer variables. By implementing this *margin* correction, CMAwM stably handles binary and integer variables while keeping the efficiency of CMA-ES for continuous variables.

To illustrate the impact of introducing the margin correction, we conducted experiments using CMA-ES with and without the margin. For simplicity, we considered a mixed-integer optimization problem, where the first d_{co} elements are continuous, and the remaining d_{bi} elements are binary. The encoding function is defined as follows:

$$\text{ENC}_f(x) = \begin{cases} x_i & (1 \leq i \leq d_{co}) \\ \mathbb{I}\{x_i > 0.5\} & (d_{co} + 1 \leq i \leq d_{co} + d_{bi}) \end{cases} \quad (9)$$

where $\mathbb{I}\{\cdot\}$ is the indicator function, and d_{co} and d_{bi} are the number of continuous and binary variables, respectively. The encoded solution is expressed as $\bar{x} = \text{ENC}_f(x)$. As an example, we employed the EllipsoidOneMax function which is defined as follows:

$$f_{\text{EllipsoidOneMax}}(\bar{x}) = \sum_{j=1}^{d_{co}} \left(1000 \frac{j-1}{d_{co}-1} \bar{x}_j \right)^2 + d_{bi} - \sum_{k=d_{co}+1}^{d_{co}+d_{bi}} \bar{x}_k.$$

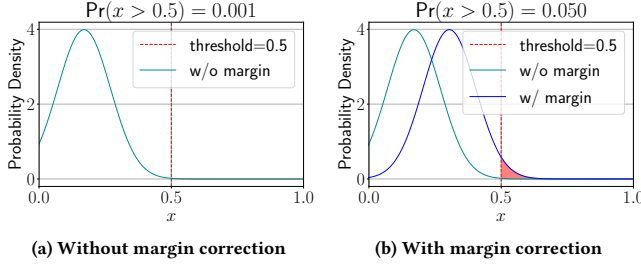


Figure 9: Motivation for CMAwM. (a) Vanilla CMA-ES, which does not perform margin correction, risks biasing search toward only 0s or 1s. (b) CMAwM introduces a lower bound on probability by margin correction, so that sample is not fixed to single discrete value.

We set the number of dimension to $d_{co} = d_{bi} = 20$. Figure 10 shows the results of the vanilla CMA-ES and CMAwM. We can observe that CMAwM successfully continues to improve the quality of the objective function value, whereas the vanilla CMA-ES has ceased to make such improvements.

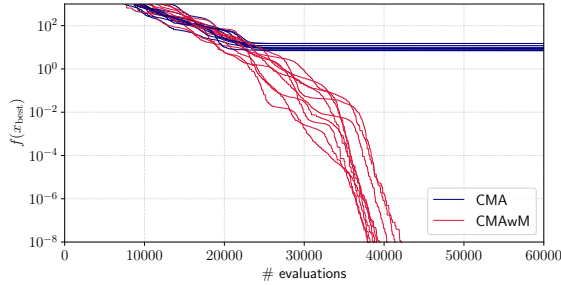


Figure 10: Comparison between vanilla CMA-ES and CMAwM on the 40-D EllipsoidOneMax function. Initial distribution is $m^{(0)} = [0.5, \dots, 0.5]$, $\sigma^{(0)} = 2.0$, $C^{(0)} = I$. Population size is set as $\lambda = 4 + \lceil 3 \log(d) \rceil = 15$. Vanilla CMA-ES stopped improving quality of objective function value, whereas CMAwM succeeds in finding optimum.

How to use? Listing 6 shows an example code using CMAwM on the 40-D EllipsoidOneMax function, which includes continuous and binary variables. The `bounds` variable defines the search space: `np.tile([-np.inf, np.inf], ...)` indicates that continuous variables are unconstrained; `np.tile([0, 1], ...)` is for binary variables. The `steps` variable specifies the discretization granularity: `np.zeros` is used for continuous variables and `np.ones` is used for binary variables. The `ask` method returns two values: `x_eval` and `x_tell`: `x_eval` is the encoded solution containing binary values, and is used for evaluation. `x_tell` is the raw solution that does not contain binary values, and is used for `tell` method. Our repository includes additional examples with integer variables.

```
1 import numpy as np
2 from cmaes import CMAwM
3
4 def ellipsoid_onemax(x_eval, binary_dim):
5     ...
6
7 bin_dim, cont_dim = 20, 20
8 dim = bin_dim + cont_dim
9 bounds = np.concatenate([
10     # for continuous variables
11     np.tile([-np.inf, np.inf], (cont_dim, 1)),
12     # for binary variables
13     np.tile([0, 1], (bin_dim, 1)),
14 ])
15 # np.zeros is for continuous, np.ones is for binary
16 steps = np.concatenate([np.zeros(cont_dim), np.ones(
17     bin_dim)])
18 # CMAwM needs bounds and steps for initialization
19 optimizer = CMAwM(mean=np.zeros(dim), sigma=2.0, bounds=
20     bounds, steps=steps)
21
22 for _ in range(10000):
23     solutions = []
24     for _ in range(optimizer.population_size):
25         x_eval, x_tell = optimizer.ask()
26         # x_eval is partially encoded in binary for eval.
27         value = ellipsoid_onemax(x_eval, binary_dim)
28         # x_tell is continuous and used for CMA update
29         solutions.append((x_tell, value))
30     optimizer.tell(solutions)
```

Listing 6: Example code of CMAwM on continuous and binary optimization problems.

5 SUMMARY AND DISCUSSIONS

This paper introduced `cmaes`, a simple yet practical CMA-ES library in Python. The library’s simplicity not only enhanced its software quality but also enabled easy integration with other libraries, such as Optuna [1]. Despite its simplicity, `cmaes` implemented highly practical methods that were recently proposed, such as LRA-CMA [37], WS-CMA [39], and CMAwM [19], all with easy-to-use APIs. We hope that our library has become the ideal starting point for practitioners interested in exploring CMA-ES.

It is important to acknowledge that the features in our library are not comprehensive. For example, `pycma` includes many features that our library lacks, as discussed in Section 1. Moreover, our library is not primarily designed for rigorous benchmarking tasks, such as comparing the performance of various optimizers. In this case, specialized software like COCO [26] is recommended.

Finally, we openly and warmly invite discussions, questions, feature requests, and contributions to our library. We believe that engaging in these interactions is crucial for identifying significant, yet unresolved, practical challenges. It is our hope that our library will serve as a bridge between research and development, significantly enhancing the practical utility of CMA-ES.

REFERENCES

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2623–2631.
- [2] Youhei Akimoto, Anne Auger, and Nikolaus Hansen. 2014. Comparison-Based Natural Gradient Optimization in High Dimension. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 373–380.

- [3] Youhei Akimoto and Nikolaus Hansen. 2016. Projection-Based Restricted Covariance Matrix Adaptation for High Dimension. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 197–204.
- [4] Youhei Akimoto and Nikolaus Hansen. 2020. Diagonal Acceleration for Covariance Matrix Adaptation Evolution Strategies. *Evolutionary Computation* 28, 3 (2020), 405–435.
- [5] Youhei Akimoto, Yuichi Nagata, Isao Ono, and Shigenobu Kobayashi. 2010. Bidirectional Relation between CMA Evolution Strategies and Natural Evolution Strategies. In *International Conference on Parallel Problem Solving from Nature*. 154–163.
- [6] Florin Andrei. 2024. Efficient Feature Selection via CMA-ES (Covariance Matrix Adaptation Evolution Strategy). <https://towardsdatascience.com/efficient-feature-selection-via-cma-es-covariance-matrix-adaptation-evolution-strategy-ee312bc7b173/>. Retrieved 17-January-2024.
- [7] Asma Atamna, Anne Auger, and Nikolaus Hansen. 2017. Linearly Convergent Evolution Strategies via Augmented Lagrangian Constraint Handling. In *Proceedings of the ACM/SIGEVO Conference on Foundations of Genetic Algorithms*. 149–161.
- [8] Anne Auger and Nikolaus Hansen. 2005. A Restart CMA Evolution Strategy With Increasing Population Size. In *IEEE Congress on Evolutionary Computation*.
- [9] Python Code Quality Authority. 2010. Flake8. <https://github.com/PyCQA/flake8/>.
- [10] Julian Blank and Kalyanmoy Deb. 2020. Pymoo: Multi-Objective Optimization in Python. *IEEE Access* 8 (2020), 89497–89509.
- [11] Yann Collette, Nikolaus Hansen, Gilles Pujol, Daniel Salazar Aponte, and Rodolphe Le Riche. 2013. Object-Oriented Programming of Optimizers—Examples in Scilab. *Multidisciplinary Design Optimization in Computational Mechanics* (2013), 499–538.
- [12] Matthias Feurer and Frank Hutter. 2019. Hyperparameter Optimization. *Automated Machine Learning: Methods, Systems, Challenges* (2019), 3–33.
- [13] Python Software Foundation. 2003. The Python Package Index. <https://pypi.org/>.
- [14] Python Software Foundation. 2018. Black. <https://github.com/psf/black/>.
- [15] Peter I Frazier. 2018. A Tutorial on Bayesian Optimization. *arXiv preprint arXiv:1807.02811* (2018).
- [16] John George, Ce Gao, Richard Liu, Hou Gang Liu, Yuan Tang, Ramdoot Pydipaty, and Amit Kumar Saha. 2020. A Scalable and Cloud-Native Hyperparameter Tuning System. *arXiv preprint arXiv:2006.02085* (2020).
- [17] Google. 2021. Atheris. <https://github.com/google/atheris/>.
- [18] David Ha and Jürgen Schmidhuber. 2018. World Models. *arXiv preprint arXiv:1803.10122* (2018).
- [19] Ryoki Hamano, Shota Saito, Masahiro Nomura, and Shinichi Shirakawa. 2022. CMA-ES with Margin: Lower-Bounding Marginal Probability for Mixed-Integer Black-Box Optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 639–647.
- [20] Ryoki Hamano, Shota Saito, Masahiro Nomura, and Shinichi Shirakawa. 2024. Marginal Probability-Based Integer Handling for CMA-ES Tackling Single- and Multi-Objective Mixed-Integer Black-Box Optimization. *ACM Transactions on Evolutionary Learning* (2024).
- [21] Nikolaus Hansen. 2009. Benchmarking a BI-Population CMA-ES on the BBOB-2009 Function Testbed. In *Proceedings of the Genetic and Evolutionary Computation Conference: Late Breaking Papers*. 2389–2396.
- [22] Nikolaus Hansen. 2016. The CMA Evolution Strategy: A Tutorial. *arXiv preprint arXiv:1604.00772* (2016).
- [23] Nikolaus Hansen. 2019. A Global Surrogate Assisted CMA-ES. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 664–672.
- [24] Nikolaus Hansen, Youhei Akimoto, and Petr Baudis. 2019. CMA-ES/pycma on Github. Zenodo, DOI:10.5281/zenodo.2559634.
- [25] Nikolaus Hansen and Anne Auger. 2013. Principled Design of Continuous Stochastic Search: From Theory to Practice. In *Theory and Principled Methods for the Design of Metaheuristics*. 145–180.
- [26] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. 2021. COCO: A Platform for Comparing Continuous Optimizers in a Black-Box Setting. *Optimization Methods and Software* 36, 1 (2021), 114–144.
- [27] Nikolaus Hansen and Stefan Kern. 2004. Evaluating the CMA Evolution Strategy on Multimodal Test Functions. In *International Conference on Parallel Problem Solving from Nature*. 282–291.
- [28] Nikolaus Hansen and Andreas Ostermeier. 2001. Completely Derandomized Self-Adaptation in Evolution Strategies. *Evolutionary computation* 9, 2 (2001), 159–195.
- [29] Charles R. Harris, K. Jarrod Millman, Stéfán J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362.
- [30] Kotaro Kikuchi, Mayu Otani, Kota Yamaguchi, and Edgar Simo-Serra. 2021. Modeling Visual Containment for Web Page Layout Optimization. In *Computer Graphics Forum*, Vol. 40. 33–44.
- [31] Kotaro Kikuchi, Edgar Simo-Serra, Mayu Otani, and Kota Yamaguchi. 2021. Constrained Graphic Layout Generation via Latent Optimization. In *Proceedings of the ACM International Conference on Multimedia*. 88–96.
- [32] Robert Tjarko Lange. 2023. evosax: JAX-based Evolution Strategies. In *Proceedings of the Genetic and Evolutionary Computation Companion*. 659–662.
- [33] David R MacIver, Zac Hatfield-Dodds, et al. 2019. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software* 4, 43 (2019), 1891.
- [34] Atsuhiko Miyagi, Youhei Akimoto, and Hajime Yamamoto. 2018. Well Placement Optimization for Carbon Dioxide Capture and Storage via CMA-ES with Mixed Integer Support. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 1696–1703.
- [35] Hidekazu Miyazawa and Youhei Akimoto. 2017. Effect of the Mean Vector Learning Rate in CMA-ES. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 721–728.
- [36] Kouhei Nishida and Youhei Akimoto. 2018. PSA-CMA-ES: CMA-ES with Population Size Adaptation. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 865–872.
- [37] Masahiro Nomura, Youhei Akimoto, and Isao Ono. 2023. CMA-ES with Learning Rate Adaptation: Can CMA-ES with Default Population Size Solve Multimodal and Noisy Problems?. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 839–847.
- [38] Masahiro Nomura, Youhei Akimoto, and Isao Ono. 2024. CMA-ES with Learning Rate Adaptation. *arXiv preprint arXiv:2401.15876* (2024).
- [39] Masahiro Nomura, Shuhei Watanabe, Youhei Akimoto, Yoshihiko Ozaki, and Masaki Onishi. 2021. Warm Starting CMA-ES for Hyperparameter Optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 9188–9196.
- [40] Lennart Oswald Purucker and Joeran Beel. 2023. CMA-ES for Post Hoc Ensembling in AutoML: A Great Success and Salvageable Failure. In *AutoML Conference 2023*.
- [41] Python. 2014. Mypy. <https://github.com/python/mypy/>.
- [42] Jeremy Rapin and Olivier Teytaud. 2018. Nevergrad - A gradient-free optimization platform. <https://GitHub.com/FacebookResearch/Nevergrad>.
- [43] Luis Miguel Rios and Nikolaos V Sahinidis. 2013. Derivative-free optimization: A review of algorithms and comparison of software implementations. *Journal of Global Optimization* 56, 3 (2013), 1247–1293.
- [44] Raymond Ros and Nikolaus Hansen. 2008. A Simple Modification in CMA-ES Achieving Linear Time and Space Complexity. In *International Conference on Parallel Problem Solving from Nature*. 296–305.
- [45] Petru Rares Sincraian. 2018. PePy. <https://www.pepy.tech/>. Retrieved 17-January-2024.
- [46] Tianxiang Sun, Zhengfu He, Hong Qian, Yunhua Zhou, Xuan-Jing Huang, and Xipeng Qiu. 2022. BBTv2: Towards a Gradient-Free Future with Large Language Models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 3916–3930.
- [47] Tianxiang Sun, Yunfan Shao, Hong Qian, Xuanjing Huang, and Xipeng Qiu. 2022. Black-Box Tuning for Language-Model-as-a-Service. In *International Conference on Machine Learning*. 20841–20855.
- [48] Yujin Tang, Yingtao Tian, and David Ha. 2022. EvoJAX: Hardware-Accelerated Neuroevolution. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 308–311.
- [49] Optuna Development Team. 2019. kurobako. <https://github.com/optuna/kurobako/>.
- [50] André Thomaser, Jacob De Nobel, Diederick Vermetten, Furong Ye, Thomas Bäck, and Anna Kononova. 2023. When to be Discrete: Analyzing Algorithm Performance on Discretized Continuous Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 856–863.
- [51] Yohei Watanabe, Kento Uchida, Ryoki Hamano, Shota Saito, Masahiro Nomura, and Shinichi Shirakawa. 2023. (1+1)-CMA-ES with Margin for Discrete and Mixed-Integer Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 882–890.
- [52] Daan Wierstra, Tom Schaul, Tobias Glasmachers, Yi Sun, Jan Peters, and Jürgen Schmidhuber. 2014. Natural Evolution Strategies. *The Journal of Machine Learning Research* 15, 1 (2014), 949–980.
- [53] Yichi Zhang, Daniel W Apley, and Wei Chen. 2020. Bayesian Optimization for Materials Design with Mixed Quantitative and Qualitative Variables. *Scientific reports* 10, 1 (2020), 4924.